

Turing machines

Exercise sheet 3 (solutions)

Exercise 3.1. Construct Turing machines computing the following functions (with numbers represented in binary): (a) $n \mapsto 2n$; (b) $n \mapsto n + 1$.

Solution. For (a), see `ex3.1a-double.yaml`; for (b), see `ex3.1b-increment.yaml`. You can copy-paste the code at <https://turingmachine.io> to examine the states, transitions and behaviour. $\square$

Exercise 3.2. Describe Turing machines that convert a natural number: (a) from unary encoding into binary encoding; (b) from binary encoding into unary encoding. (In unary encoding, $n > 0$ is represented by n consecutive 1s, and 0 by ε).

Solution. For (a), see `ex3.2a-unary-to-binary.yaml`; for (b), see `ex3.2b-binary-to-unary.yaml`. $\square$

Exercise 3.3. Describe a Turing machine that computes the length of an input word over an arbitrary alphabet $\Sigma \supseteq \{0, 1\}$, writing the answer in binary on the tape.

Solution. See `ex3.3-length.yaml`. $\square$

Exercise 3.4. Describe Turing machines computing the addition function $\langle n, m \rangle \mapsto n + m$, using: (a) unary encoding; (b) binary encoding.

Solution. (a) See `ex3.4a-unary-add.yaml`. See also Example 29 in Lecture 3.

(b) Write the input as two binary numbers separated by $+$:

$$\dots \sqcup \underbrace{a_k \dots a_0}_n + \underbrace{b_l \dots b_0}_m \sqcup \dots$$

While m is non-zero, decrement m and increment n , using the usual binary borrow and carry procedures. When $m = 0$, erase its representation and the separator, place the head on the first bit of n , and halt.

$\square$

Exercise 3.5. Describe a Turing machine computing the multiplication function $\langle n, m \rangle \mapsto n \cdot m$ using unary encoding.

Solution. Write the input as two blocks of 1s separated by $*$:

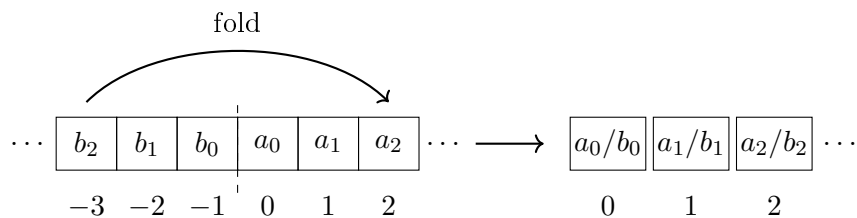
$$\dots \sqcup \underbrace{11\dots 1}_n * \underbrace{11\dots 1}_m \sqcup \dots$$

If either block is empty, erase the tape and halt. Otherwise, place a delimiter $=$ after the second block and construct the product to its right.

Repeatedly mark one unmarked 1 in the first block with X . For each such mark, scan the second block; mark each of its 1s temporarily with Y and append one 1 to the product. On reaching $=$, restore all Y s to 1 and begin the next pass. Once every symbol in the first block is marked, erase the input, markers and delimiters. The remaining block contains nm ones; halt on its first symbol. $\square$

Exercise 3.6. Prove that the following versions of Turing machines are equivalent to the original in computational power: (a) machines with a tape that is infinite only in one direction; (b) machines with several tapes and several read/write heads; (c) machines that never stay put—every transition shifts them either right or left on the tape.

Solution. (a) **Semi-infinite tape.** We show how to simulate the standard tape with just a semi-infinite one. Imagine that the cells of the standard tape are indexed by integers, and that the machine originally starts from cell 0. Fold the tape at the border between cells -1 and 0. Cell i of the semi-infinite tape then stores the contents of original cells i and $-i - 1$:



Formally, we write ‘double symbols’ a/b : cell i stores a , while cell $-i - 1$ stores b . The finite control records which track is active: for each state q , we create two copies, in one the machine behaves as if reading the symbol before $/$, in the other—as if reading the one after, with the direction of movement reversed in the second case. An attempt to move between original cells 0 and -1 switches between the two modes.

(b) **Several tapes.** To simulate k tapes with one (for a fixed k), use ‘large’ symbols $(a_1/\dots/a_k)$. Use a marked version of a component to show the position of its head. Then go around with the actual head and move these simulated heads according to the transitions.

(c) **No stay-put move.** Replace

$$\delta(q, a) = (q', b, S)$$

by a move left to a fresh state, followed by a move right to q' . The second transition preserves whichever tape symbol it reads. The combined effect writes b , returns the head to the original cell, and enters q' . $\square$

Exercise 3.7. Describe how a Turing machine could simulate gadgets of modern programming languages: (a) **for** loops; (b) **while** loops; (c) function composition; (d) recursion.

Solution. (a) **for loop.** Store the number of remaining iterations in a dedicated tape region.

Before each iteration, run a machine that checks this number. If it is zero, halt; otherwise, enter the initial state of the machine for the loop body. When the body would halt, enter a machine that decreases the counter by one, then return to the initial state of the checking machine.

(b) **while loop.** Run an accept-reject machine for the condition. Connect its rejecting state to the halting state, and its accepting state to the initial state of the machine for the loop body. When the body would halt, return to the initial state of the condition-checking machine.

(c) **Composition.** Suppose you want to compute $g(f(w))$, and you have machines for f and g . Run the machine for f on w , but connect the state in which it would halt to the initial state of the machine for g . The latter starts with $f(w)$ on the tape and produces the final output $g(f(w))$.

(d) **Recursion.** The tape serves as a stack of workspaces. Reserve part of the tape for the information needed to resume an unfinished computation. Before starting a recursive computation, record that information and its arguments there. When it finishes, restore the most recently recorded information and continue from that point. A base case finishes without starting another recursive computation. $\square$

Exercise 3.8. A machine is called a *tape cleaner* if, started in any configuration, it eventually stops in the configuration $q_0\sqcup$ (i.e. it cleans all the content off the tape and stops). Prove that such a machine does not exist.

Solution. Suppose that a tape cleaner C exists. On an empty tape it halts after some k steps, having inspected only cells at distance at most k from its starting position.

Now place a single 1 at distance $k + 1$, leaving every nearer cell blank. For the first k steps, C sees exactly what it saw on the empty tape, so it performs the same computation and halts. The distant 1 remains unerased. $\square$

Exercise 3.9. Describe (in broad strokes) a suitable encoding of Turing machines as binary words $[M] \in \{0, 1\}^*$ and a *universal Turing machine* U such that, for any machine M computing the function f_M , and any input word w , $f_U(\langle [M], w \rangle) = f_M(w)$.

Solution. See the section “The universal Turing machine” in Lecture 3. $\square$

Exercise 3.10. What can be said about a computation if the same configuration (tape contents, head position and machine state) appears twice during its execution?

Solution. The machine will never halt. A configuration determines its unique successor, so after returning to an earlier configuration the machine repeats the same finite sequence of configurations forever. $\square$

Exercise 3.11. Let $f: \{0,1\}^* \rightarrow \mathbb{N}$ be the partial function defined as follows: $f(\langle M, x \rangle) = k$ if the Turing machine M on input x visits the blank symbol $\sqcup$ for the last time at step k . If no such step k exists (that is, if M visits $\sqcup$ infinitely often, or never visits $\sqcup$ at all), then $f(\langle M, x \rangle)$ is undefined. Prove that the function f is computable.

Solution. Simulate M , recording the current step and the most recent step at which it scanned $\sqcup$. Also record every configuration reached since that visit, clearing the record whenever $\sqcup$ is scanned again.

If M halts, output the recorded step, if one exists; otherwise loop forever. If a recorded configuration repeats before another blank is scanned, that same cycle will repeat without visiting a blank. Output the recorded step, or loop forever if no blank has yet been scanned.

If there is a last visit and M does not halt, choose a finite interval containing the head and every non-blank cell immediately afterwards. The head cannot leave this interval without scanning a blank. Only finitely many configurations are possible within it, so one must repeat. Thus the procedure outputs exactly the last-visit step when it exists and loops forever otherwise. $\square$